GPU SYSTEMS STUDY GUIDE

LeetGPU Complete GPU Kernel Handbook

All listed Easy, Medium and Hard challenges, with mathematics, PyTorch reference code, CUDA core-kernel sketches, thread mapping, memory traffic, hardware execution, optimization guidance and topic diagrams.

Educational edition. Advanced kernels show the essential algorithmic structure; production kernels require architecture-specific tuning, boundary handling and validation.

How to read each problem

Framework side

PyTorch expresses tensor semantics, shapes and numerical behavior.

Kernel side

CUDA exposes the thread mapping, memory hierarchy, synchronization and data reuse.

Hardware side

Warps are issued on an SM; operands travel from HBM through cache/shared memory into registers and execution units.

Performance side

Decide whether the kernel is bandwidth-, latency-, synchronization-, or compute-bound.

Universal hardware path



Important: PyTorch code is the correctness reference. CUDA snippets are minimal core kernels or structural sketches intended to reveal the algorithm. They are not drop-in replacements for cuBLAS, cuDNN, CUTLASS, or fused production libraries.

Table of contents

001. **Vector Addition** Easy
002. **Matrix Multiplication** Easy
003. **Matrix Transpose** Easy
004. **Color Inversion** Easy
005. **Matrix Addition** Easy
006. **1D Convolution** Easy
007. **Reverse Array** Easy
008. **ReLU** Easy
009. **Leaky ReLU** Easy
010. **Rainbow Table** Easy
011. **Matrix Copy** Easy
012. **Simple Inference** Easy
013. **Sigmoid Linear Unit** Easy
014. **Swish-Gated Linear Unit** Easy
015. **Value Clipping** Easy
016. **Interleave Arrays** Easy
017. **Gaussian Error Gated Linear Unit** Easy
018. **RGB to Grayscale** Easy
019. **Sigmoid Activation** Easy
020. **GELU Activation** Easy
021. **Reduction** Medium
022. **Softmax** Medium
023. **Softmax Attention** Medium
024. **2D Convolution** Medium
025. **3D Convolution** Medium
026. **Histogramming** Medium
027. **Prefix Sum** Medium
028. **Dot Product** Medium
029. **Sparse Matrix-Vector Multiplication** Medium
030. **General Matrix Multiplication (GEMM)** Medium
031. **Categorical Cross Entropy Loss** Medium
032. **Mean Squared Error** Medium
033. **Gaussian Blur** Medium
034. **Top K Selection** Medium
035. **Batched Matrix Multiplication** Medium
036. **INT8 Quantized MatMul** Medium
037. **Ordinary Least Squares** Medium
038. **Logistic Regression** Medium
039. **Monte Carlo Integration** Medium
040. **Matrix Power** Medium
041. **Nearest Neighbor** Medium
042. **Batch Normalization** Medium
043. **2D Max Pooling** Medium
044. **Count Array Element** Medium
045. **Count 2D Array Element** Medium
046. **Count 3D Array Element** Medium
047. **Subarray Sum** Medium
048. **2D Subarray Sum** Medium
049. **3D Subarray Sum** Medium
050. **RMS Normalization** Medium
051. **Max Subarray Sum** Medium
052. **Attention with Linear Biases** Medium
053. **FP16 Batched Matrix Multiplication** Medium
054. **FP16 Dot Product** Medium
055. **Top-p Sampling** Medium
056. **Rotary Positional Embedding** Medium
057. **Weight Dequantization** Medium
058. **MoE Top-K Gating** Medium
059. **2D Jacobi Stencil** Medium
060. **Segmented Exclusive Prefix Sum** Medium
061. **Parallel Merge** Medium
062. **Stream Compaction** Medium
063. **Sparse Matrix-Dense Matrix Multiplication** Medium
064. **Adder Transformer Inference** Medium
065. **Integral Image** Medium
066. **2D FFT** Medium
067. **Connected Components** Medium
068. **Grouped Query Attention** Medium

069. **INT4 Weight-Only Quantized MatMul** Medium
070. **Linear Recurrence** Medium
071. **Fused Residual Add and RMS Norm** Medium
072. **SwiGLU MLP Block** Medium
073. **LoRA Linear** Medium
074. **Paged KV-Cache Attention** Medium
075. **Speculative Decoding Verification** Medium
076. **Shared-Prefix Cascade Attention** Medium
077. **Causal Depthwise Conv1d** Medium
078. **MoE Token Dispatch** Medium
079. **Decaying Causal Attention** Medium
080. **SSM Selective Scan** Medium
081. **INT8 KV-Cache Attention** Medium
082. **Beam Search Step** Medium
083. **Grouped GEMM** Medium
084. **N-body Gravitational Force** Medium
085. **Variable-Length Causal Attention** Medium
086. **Hadamard Transform** Medium
087. **Min-P Sampling** Medium
088. **Group Normalization** Medium
089. **Token Embedding Layer** Medium
090. **PPO Clipped Surrogate Loss** Medium
091. **DPO Sequence Loss** Medium
092. **GRPO Surrogate Loss** Medium
093. **Parallel Reverse Scan (GAE)** Medium
094. **Softmax Attention Backward** Medium
095. **Attention with Sinks** Medium
096. **Layer Normalization** Medium
097. **Fused QKV Projection with RoPE and KV Cache Update** Medium
098. **Vision Transformer Patch Embedding** Medium
099. **Kimi Delta Attention** Medium
100. **Fused Logit Penalties** Medium
101. **Scatter Add** Medium
102. **Row-wise Argmax** Medium
103. **Fused AdamW** Medium
104. **Split-KV Attention Reduction** Medium
105. **Multi-Head Attention** Hard
106. **Multi-Agent Simulation** Hard
107. **Sorting** Hard
108. **K-Means Clustering** Hard
109. **Multi-Head Cross-Attention** Hard
110. **Radix Sort** Hard
111. **Fast Fourier Transform** Hard
112. **BFS Shortest Path** Hard
113. **Causal Self-Attention** Hard
114. **Linear Self-Attention** Hard
115. **Sliding Window Self-Attention** Hard
116. **All-Pairs Shortest Paths** Hard
117. **GPT-2 Transformer Block** Hard
118. **NVFP4 Matrix Multiplication** Hard
119. **Llama Transformer Block** Hard
120. **TurboQuant KV Cache Attention** Hard
121. **Multi-Head Latent Attention Decode** Hard
122. **Diffusion Transformer Block** Hard
123. **Non-Maximum Suppression** Hard
124. **Block-Sparse KV Selection Attention** Hard
125. **Multi-Scale Deformable Attention** Hard

FOUNDATION

Grid, block, warp, thread

A kernel launch creates a grid. Each block is placed on an SM. Warps of 32 threads execute a common instruction stream. Divergent branches serialize paths.



Question	What to inspect
Where is reuse?	Registers, shared-memory tiles, L2 residency.
Where does synchronization happen?	Warp shuffle, block barrier, atomic, or multi-kernel boundary.
What limits speed?	Bytes moved, instructions, dependency latency, imbalance, launch overhead.

FOUNDATION

Memory hierarchy

Registers are thread-private; shared memory is block-local; L1/L2 cache reduce HBM traffic. Coalesced warp accesses combine adjacent words into fewer memory transactions.



Question	What to inspect
Where is reuse?	Registers, shared-memory tiles, L2 residency.
Where does synchronization happen?	Warp shuffle, block barrier, atomic, or multi-kernel boundary.
What limits speed?	Bytes moved, instructions, dependency latency, imbalance, launch overhead.

FOUNDATION

Performance model

Elementwise kernels are usually bandwidth-bound. GEMM and attention can become compute-bound when tiles reuse operands. Occupancy helps hide latency but does not replace locality.



Question	What to inspect
Where is reuse?	Registers, shared-memory tiles, L2 residency.
Where does synchronization happen?	Warp shuffle, block barrier, atomic, or multi-kernel boundary.
What limits speed?	Bytes moved, instructions, dependency latency, imbalance, launch overhead.

FOUNDATION

Numerical behavior

Parallel sums are order-dependent. Softmax uses max-subtraction. FP16/BF16 inputs usually accumulate in FP32. Quantized kernels require explicit scales, zero points and saturation.



Question	What to inspect
Where is reuse?	Registers, shared-memory tiles, L2 residency.
Where does synchronization happen?	Warp shuffle, block barrier, atomic, or multi-kernel boundary.
What limits speed?	Bytes moved, instructions, dependency latency, imbalance, launch overhead.

PROBLEM 001 · ELEMENTWISE

Easy

Vector Addition

$$y[i] = f(x[i])$$



What the topic does

Vector Addition maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 002 · GEMM

Easy

Matrix Multiplication

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Matrix Multiplication maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 003 · ELEMENTWISE

Easy

Matrix Transpose

$$y[i] = f(x[i])$$



What the topic does

Matrix Transpose maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 004 · ELEMENTWISE

Easy

Color Inversion

$$y[i] = f(x[i])$$



What the topic does

Color Inversion maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 005 · ELEMENTWISE

Easy

Matrix Addition

$$y[i] = f(x[i])$$



What the topic does

Matrix Addition maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 006 · STENCIL

Easy

1D Convolution

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

1D Convolution maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 007 · ELEMENTWISE

Easy

Reverse Array

$$y[i] = f(x[i])$$



What the topic does

Reverse Array maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Easy

ReLU

$$y[i] = f(x[i])$$



What the topic does

ReLU maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 009 · ELEMENTWISE

Easy

Leaky ReLU

$$y[i] = f(x[i])$$



What the topic does

Leaky ReLU maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 010 · ELEMENTWISE

Easy

Rainbow Table

$$y[i] = f(x[i])$$



What the topic does

Rainbow Table maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Easy

Matrix Copy

$$y[i] = f(x[i])$$



What the topic does

Matrix Copy maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 012 · MODEL

Easy

Simple Inference

$$y = block(x; parameters, state)$$



What the topic does

Simple Inference maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 013 · GEMM

Easy

Sigmoid Linear Unit

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Sigmoid Linear Unit maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 014 · GEMM

Easy

Swish-Gated Linear Unit

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Swish-Gated Linear Unit maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Easy

Value Clipping

$$y[i] = f(x[i])$$



What the topic does

Value Clipping maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 016 · ELEMENTWISE

Easy

Interleave Arrays

$$y[i] = f(x[i])$$



What the topic does

Interleave Arrays maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 017 · GEMM

Easy

Gaussian Error Gated Linear Unit

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Gaussian Error Gated Linear Unit maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 018 · ELEMENTWISE

Easy

RGB to Grayscale

$$y[i] = f(x[i])$$



What the topic does

RGB to Grayscale maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 019 · ELEMENTWISE

Easy

Sigmoid Activation

$$y[i] = f(x[i])$$



What the topic does

Sigmoid Activation maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 020 · ELEMENTWISE

Easy

GELU Activation

$$y[i] = f(x[i])$$



What the topic does

GELU Activation maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Reduction

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Reduction maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Softmax

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Softmax maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 023 · ATTENTION

Medium

Softmax Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Softmax Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 024 · STENCIL

Medium

2D Convolution

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

2D Convolution maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 025 · STENCIL

Medium

3D Convolution

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

3D Convolution maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Histogramming

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Histogramming maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 027 · SCAN

Medium

Prefix Sum

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Prefix Sum maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Dot Product

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Dot Product maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 029 · ELEMENTWISE

Medium

Sparse Matrix-Vector Multiplication

$$y[i] = f(x[i])$$



What the topic does

Sparse Matrix-Vector Multiplication maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 030 · GEMM

Medium

General Matrix Multiplication (GEMM)

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

General Matrix Multiplication (GEMM) maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 031 · REDUCTION

Medium

Categorical Cross Entropy Loss

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Categorical Cross Entropy Loss maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Mean Squared Error

$$y[i] = f(x[i])$$



What the topic does

Mean Squared Error maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 033 · STENCIL

Medium

Gaussian Blur

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

Gaussian Blur maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Top K Selection

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Top K Selection maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 035 · GEMM

Medium

Batched Matrix Multiplication

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Batched Matrix Multiplication maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 036 · GEMM

Medium

INT8 Quantized MatMul

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

INT8 Quantized MatMul maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Ordinary Least Squares

$$y[i] = f(x[i])$$



What the topic does

Ordinary Least Squares maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Logistic Regression

$$y[i] = f(x[i])$$



What the topic does

Logistic Regression maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Monte Carlo Integration

$$y[i] = f(x[i])$$



What the topic does

Monte Carlo Integration maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Matrix Power

$$y[i] = f(x[i])$$



What the topic does

Matrix Power maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 041 · GRAPH

Medium

Nearest Neighbor

$$state[v] = aggregate(\{state[u] : u \text{ in } N(v)\})$$



What the topic does

Nearest Neighbor maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Threads process vertices, edges, points or bodies. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
# Vectorized pairwise or scatter formulation
out = aggregate(src[index])
```

CUDA core kernel / structural sketch

```
__global__ void graph_step(const int* row,const int* col,float* state){
  int v=blockIdx.x*blockDim.x+threadIdx.x;
  for(int e=row[v];e<row[v+1];e++) state[v]=combine(state[v],state[col[e]]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

load imbalance, atomics, irregular memory.

Optimization direction

frontier compaction; CSR layout; shared tiling; minimize atomic contention.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 042 · REDUCTION

Medium

Batch Normalization

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Batch Normalization maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 043 · STENCIL

Medium

2D Max Pooling

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

2D Max Pooling maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 044 · REDUCTION

Medium

Count Array Element

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Count Array Element maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 045 · REDUCTION

Medium

Count 2D Array Element

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Count 2D Array Element maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 046 · REDUCTION

Medium

Count 3D Array Element

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Count 3D Array Element maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 047 · REDUCTION

Medium

Subarray Sum

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Subarray Sum maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 048 · REDUCTION

Medium

2D Subarray Sum

$$r = \text{reduce}_i\ f(x_i)$$



What the topic does

2D Subarray Sum maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 049 · REDUCTION

Medium

3D Subarray Sum

$$r = \text{reduce_i } f(x_i)$$



What the topic does

3D Subarray Sum maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 050 · REDUCTION

Medium

RMS Normalization

$$r = \text{reduce_i } f(x_i)$$



What the topic does

RMS Normalization maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 051 · REDUCTION

Medium

Max Subarray Sum

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Max Subarray Sum maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Attention with Linear Biases

$$O = \text{softmax}(Q\ K^{\wedge}T / \text{sqrt}(d) + \text{mask})\ V$$



What the topic does

Attention with Linear Biases maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 053 · GEMM

Medium

FP16 Batched Matrix Multiplication

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

FP16 Batched Matrix Multiplication maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 054 · REDUCTION

Medium

FP16 Dot Product

$$r = \text{reduce_i } f(x_i)$$



What the topic does

FP16 Dot Product maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Top-p Sampling

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Top-p Sampling maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 056 · MODEL

Medium

Rotary Positional Embedding

$$y = block(x; parameters, state)$$



What the topic does

Rotary Positional Embedding maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 057 · QUANT

Medium

Weight Dequantization

$$Y = scale_A\ scale_B * (Q_A\ Q_B)$$



What the topic does

Weight Dequantization maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Integer/tiny-float MMA with scaled accumulation. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
Y = (qA.float()*sA) @ (qB.float()*sB)
```

CUDA core kernel / structural sketch

```
__global__ void dequant_matmul(const int8_t* A,const int8_t* B,const float* s,float* C){
  // Unpack/dequantize a tile, perform integer or mixed-precision MMA,
  // apply group scales, and write an FP16/FP32 output tile.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

packing, saturation, scale loads.

Optimization direction

pack weights; vectorized dequant; per-group scales; tensor-core MMA.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

MoE Top-K Gating

$$y[i] = f(x[i])$$



What the topic does

MoE Top-K Gating maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 059 · STENCIL

Medium

2D Jacobi Stencil

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

2D Jacobi Stencil maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Segmented Exclusive Prefix Sum

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Segmented Exclusive Prefix Sum maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Parallel Merge

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Parallel Merge maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Stream Compaction

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Stream Compaction maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 063 · GEMM

Medium

Sparse Matrix-Dense Matrix Multiplication

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Sparse Matrix-Dense Matrix Multiplication maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 064 · MODEL

Medium

Adder Transformer Inference

$$y = block(x; parameters, state)$$



What the topic does

Adder Transformer Inference maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Integral Image

$$y[i] = f(x[i])$$



What the topic does

Integral Image maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

2D FFT

$$X[k] = \sum_n x[n] \exp(-j\ 2\ \pi\ k\ n\ /\ N)$$



What the topic does

2D FFT maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Butterfly stages exchange power-of-two partners. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
Y = torch.fft.fft(x)
```

CUDA core kernel / structural sketch

```
__global__ void butterfly(cfloat* x,int stride){
  int i=blockIdx.x*blockDim.x+threadIdx.x; int j=i^stride;
  if(i<j){ cfloat a=x[i], b=x[j]*twiddle(i,stride); x[i]=a+b; x[j]=a-b; }
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

strided accesses and synchronization.

Optimization direction

shared-memory butterflies; radix decomposition; transpose between dimensions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 067 · GRAPH

Medium

Connected Components

$$state[v] = aggregate(\{state[u] : u \text{ in } N(v)\})$$



What the topic does

Connected Components maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Threads process vertices, edges, points or bodies. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
# Vectorized pairwise or scatter formulation
out = aggregate(src[index])
```

CUDA core kernel / structural sketch

```
__global__ void graph_step(const int* row,const int* col,float* state){
  int v=blockIdx.x*blockDim.x+threadIdx.x;
  for(int e=row[v];e<row[v+1];e++) state[v]=combine(state[v],state[col[e]]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

load imbalance, atomics, irregular memory.

Optimization direction

frontier compaction; CSR layout; shared tiling; minimize atomic contention.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Grouped Query Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Grouped Query Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 069 · GEMM

Medium

INT4 Weight-Only Quantized MatMul

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

INT4 Weight-Only Quantized MatMul maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Linear Recurrence

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Linear Recurrence maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 071 · REDUCTION

Medium

Fused Residual Add and RMS Norm

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Fused Residual Add and RMS Norm maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 072 · MODEL

Medium

SwiGLU MLP Block

$$y = block(x; parameters, state)$$



What the topic does

SwiGLU MLP Block maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 073 · GEMM

Medium

LoRA Linear

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

LoRA Linear maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Paged KV-Cache Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Paged KV-Cache Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 075 · ELEMENTWISE

Medium

Speculative Decoding Verification

$$y[i] = f(x[i])$$



What the topic does

Speculative Decoding Verification maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 076 · ATTENTION

Medium

Shared-Prefix Cascade Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Shared-Prefix Cascade Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 077 · STENCIL

Medium

Causal Depthwise Conv1d

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

Causal Depthwise Conv1d maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 078 · ELEMENTWISE

Medium

MoE Token Dispatch

$$y[i] = f(x[i])$$



What the topic does

MoE Token Dispatch maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 079 · ATTENTION

Medium

Decaying Causal Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Decaying Causal Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

SSM Selective Scan

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

SSM Selective Scan maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 081 · ATTENTION

Medium

INT8 KV-Cache Attention

$$O = \text{softmax}(Q\ K^{\wedge}T / \text{sqrt}(d) + \text{mask})\ V$$



What the topic does

INT8 KV-Cache Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 082 · ELEMENTWISE

Medium

Beam Search Step

$$y[i] = f(x[i])$$



What the topic does

Beam Search Step maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Grouped GEMM

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

Grouped GEMM maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 084 · GRAPH

Medium

N-body Gravitational Force

$$state[v] = aggregate(\{state[u] : u \text{ in } N(v)\})$$



What the topic does

N-body Gravitational Force maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Threads process vertices, edges, points or bodies. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
# Vectorized pairwise or scatter formulation
out = aggregate(src[index])
```

CUDA core kernel / structural sketch

```
__global__ void graph_step(const int* row,const int* col,float* state){
  int v=blockIdx.x*blockDim.x+threadIdx.x;
  for(int e=row[v];e<row[v+1];e++) state[v]=combine(state[v],state[col[e]]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

load imbalance, atomics, irregular memory.

Optimization direction

frontier compaction; CSR layout; shared tiling; minimize atomic contention.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Variable-Length Causal Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Variable-Length Causal Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 086 · TRANSFORM

Medium

Hadamard Transform

$$X[k] = \sum_n x[n] \exp(-j\ 2\ \pi\ k\ n\ /\ N)$$



What the topic does

Hadamard Transform maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Butterfly stages exchange power-of-two partners. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
Y = torch.fft.fft(x)
```

CUDA core kernel / structural sketch

```
__global__ void butterfly(cfloat* x,int stride){
  int i=blockIdx.x*blockDim.x+threadIdx.x; int j=i^stride;
  if(i<j){ cfloat a=x[i], b=x[j]*twiddle(i,stride); x[i]=a+b; x[j]=a-b; }
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

strided accesses and synchronization.

Optimization direction

shared-memory butterflies; radix decomposition; transpose between dimensions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 087 · SCAN

Medium

Min-P Sampling

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Min-P Sampling maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 088 · REDUCTION

Medium

Group Normalization

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Group Normalization maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 089 · MODEL

Medium

Token Embedding Layer

$$y = block(x; parameters, state)$$



What the topic does

Token Embedding Layer maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 090 · REDUCTION

Medium

PPO Clipped Surrogate Loss

$$r = \text{reduce_i } f(x_i)$$



What the topic does

PPO Clipped Surrogate Loss maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 091 · REDUCTION

Medium

DPO Sequence Loss

$$r = \text{reduce_i } f(x_i)$$



What the topic does

DPO Sequence Loss maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 092 · REDUCTION

Medium

GRPO Surrogate Loss

$$r = \text{reduce_i } f(x_i)$$



What the topic does

GRPO Surrogate Loss maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Parallel Reverse Scan (GAE)

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Parallel Reverse Scan (GAE) maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 094 · ATTENTION

Medium

Softmax Attention Backward

$$O = \text{softmax}(Q\ K^{\wedge}T\ /\ \text{sqrt}(d) + \text{mask})\ V$$



What the topic does

Softmax Attention Backward maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Attention with Sinks

$$O = \text{softmax}(Q\ K^T / \sqrt{d} + \text{mask})\ V$$



What the topic does

Attention with Sinks maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 096 · REDUCTION

Medium

Layer Normalization

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Layer Normalization maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Fused QKV Projection with RoPE and KV Cache Update

$$O = \text{softmax}(Q\ K^{\wedge}T / \text{sqrt}(d) + \text{mask})\ V$$



What the topic does

Fused QKV Projection with RoPE and KV Cache Update maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 098 · STENCIL

Medium

Vision Transformer Patch Embedding

$$y[p] = \sum_d x[p+d]\, w[d]$$



What the topic does

Vision Transformer Patch Embedding maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One thread computes one output neighborhood. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.nn.functional.conv1d(x, w, padding="same")
```

CUDA core kernel / structural sketch

```
__global__ void stencil(const float* x,const float* w,float* y,int n,int R){
  extern __shared__ float tile[];
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  tile[threadIdx.x+R]=(i<n)?x[i]:0;
  __syncthreads();
  float acc=0; for(int d=-R;d<=R;d++) acc += tile[threadIdx.x+R+d]*w[d+R];
  if(i<n) y[i]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

halo loads and reuse.

Optimization direction

shared-memory tile plus halo; avoid redundant global reads.

PyTorch vs CUDA comparison

View	PyTorch	CUDA

Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 099 · ATTENTION

Medium

Kimi Delta Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Kimi Delta Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 100 · ELEMENTWISE

Medium

Fused Logit Penalties

$$y[i] = f(x[i])$$



What the topic does

Fused Logit Penalties maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Medium

Scatter Add

$$y[i] = f(x[i])$$



What the topic does

Scatter Add maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 102 · REDUCTION

Medium

Row-wise Argmax

$$r = \text{reduce_i } f(x_i)$$



What the topic does

Row-wise Argmax maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Each block reduces a chunk; warps finish partials. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
r = torch.sum(x, dim=-1)  # replace with max/norm/loss as needed
```

CUDA core kernel / structural sketch

```
__global__ void reduce_sum(const float* x,float* out,int n){
  float v=0; for(int i=blockIdx.x*blockDim.x+threadIdx.x;i<n;i+=gridDim.x*blockDim.x) v+=x[i];
  for(int d=16;d>0;d>>=1) v += __shfl_down_sync(0xffffffff,v,d);
  if((threadIdx.x&31)==0) atomicAdd(out,v);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

synchronization and numerical stability.

Optimization direction

warp shuffles; stable max-subtraction; fuse passes.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 103 · MODEL

Medium

Fused AdamW

$$y = block(x; parameters, state)$$



What the topic does

Fused AdamW maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 104 · ATTENTION

Medium

Split-KV Attention Reduction

$$O = \text{softmax}(Q\ K^{\wedge}T / \text{sqrt}(d) + \text{mask})\ V$$



What the topic does

Split-KV Attention Reduction maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 105 · ATTENTION

Hard

Multi-Head Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Multi-Head Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 106 · GRAPH

Hard

Multi-Agent Simulation

$$state[v] = aggregate(\{state[u] : u \text{ in } N(v)\})$$



What the topic does

Multi-Agent Simulation maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Threads process vertices, edges, points or bodies. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
# Vectorized pairwise or scatter formulation
out = aggregate(src[index])
```

CUDA core kernel / structural sketch

```
__global__ void graph_step(const int* row,const int* col,float* state){
  int v=blockIdx.x*blockDim.x+threadIdx.x;
  for(int e=row[v];e<row[v+1];e++) state[v]=combine(state[v],state[col[e]]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

load imbalance, atomics, irregular memory.

Optimization direction

frontier compaction; CSR layout; shared tiling; minimize atomic contention.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Sorting

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Sorting maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 108 · GRAPH

Hard

K-Means Clustering

$$state[v] = aggregate(\{state[u] : u \text{ in } N(v)\})$$



What the topic does

K-Means Clustering maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Threads process vertices, edges, points or bodies. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
# Vectorized pairwise or scatter formulation
out = aggregate(src[index])
```

CUDA core kernel / structural sketch

```
__global__ void graph_step(const int* row,const int* col,float* state){
  int v=blockIdx.x*blockDim.x+threadIdx.x;
  for(int e=row[v];e<row[v+1];e++) state[v]=combine(state[v],state[col[e]]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

load imbalance, atomics, irregular memory.

Optimization direction

frontier compaction; CSR layout; shared tiling; minimize atomic contention.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Multi-Head Cross-Attention

$$O = \text{softmax}(Q\ K^{\wedge}T / \text{sqrt}(d) + \text{mask})\ V$$



What the topic does

Multi-Head Cross-Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Radix Sort

$$y[i] = \text{combine}(x[0:i])$$



What the topic does

Radix Sort maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Upsweep/downsweep or local scan plus block offsets. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = torch.cumsum(x, dim=-1)
```

CUDA core kernel / structural sketch

```
__global__ void scan_block(const float* x,float* y){
  __shared__ float s[BLOCK]; int t=threadIdx.x, i=blockIdx.x*BLOCK+t; s[t]=x[i];
  for(int o=1;o<BLOCK;o<<=1){ float v=t>=o?s[t-o]:0; __syncthreads(); s[t]+=v; __syncthreads(); }
  y[i]=s[t];
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

global ordering and irregular writes.

Optimization direction

warp scan; hierarchical block scan; compact with prefix positions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 111 · TRANSFORM

Hard

Fast Fourier Transform

$$X[k] = \sum_n x[n] \exp(-j\ 2\ \pi\ k\ n\ /\ N)$$



What the topic does

Fast Fourier Transform maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Butterfly stages exchange power-of-two partners. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
Y = torch.fft.fft(x)
```

CUDA core kernel / structural sketch

```
__global__ void butterfly(cfloat* x,int stride){
  int i=blockIdx.x*blockDim.x+threadIdx.x; int j=i^stride;
  if(i<j){ cfloat a=x[i], b=x[j]*twiddle(i,stride); x[i]=a+b; x[j]=a-b; }
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

strided accesses and synchronization.

Optimization direction

shared-memory butterflies; radix decomposition; transpose between dimensions.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

BFS Shortest Path

$$state[v] = aggregate(\{state[u] : u \text{ in } N(v)\})$$



What the topic does

BFS Shortest Path maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Threads process vertices, edges, points or bodies. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
# Vectorized pairwise or scatter formulation
out = aggregate(src[index])
```

CUDA core kernel / structural sketch

```
__global__ void graph_step(const int* row,const int* col,float* state){
  int v=blockIdx.x*blockDim.x+threadIdx.x;
  for(int e=row[v];e<row[v+1];e++) state[v]=combine(state[v],state[col[e]]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

load imbalance, atomics, irregular memory.

Optimization direction

frontier compaction; CSR layout; shared tiling; minimize atomic contention.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Causal Self-Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Causal Self-Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Linear Self-Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Linear Self-Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 115 · ATTENTION

Hard

Sliding Window Self-Attention

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Sliding Window Self-Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

All-Pairs Shortest Paths

$$state[v] = aggregate(\{state[u] : u \text{ in } N(v)\})$$



What the topic does

All-Pairs Shortest Paths maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Threads process vertices, edges, points or bodies. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
# Vectorized pairwise or scatter formulation
out = aggregate(src[index])
```

CUDA core kernel / structural sketch

```
__global__ void graph_step(const int* row,const int* col,float* state){
  int v=blockIdx.x*blockDim.x+threadIdx.x;
  for(int e=row[v];e<row[v+1];e++) state[v]=combine(state[v],state[col[e]]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

load imbalance, atomics, irregular memory.

Optimization direction

frontier compaction; CSR layout; shared tiling; minimize atomic contention.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 117 · MODEL

Hard

GPT-2 Transformer Block

$$y = block(x; parameters, state)$$



What the topic does

GPT-2 Transformer Block maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

NVFP4 Matrix Multiplication

$$C[m,n] = \text{sum_k } A[m,k]\ B[k,n]$$



What the topic does

NVFP4 Matrix Multiplication maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One CTA computes an output tile. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
C = A @ B  # or torch.nn.functional.linear(X, W, b)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_gemm(const half* A,const half* B,float* C,int M,int N,int K){
  __shared__ half As[T][T], Bs[T][T];
  int r=blockIdx.y*T+threadIdx.y, c=blockIdx.x*T+threadIdx.x;
  float acc=0;
  for(int k0=0;k0<K;k0+=T){
    As[threadIdx.y][threadIdx.x]=A[r*K+k0+threadIdx.x];
    Bs[threadIdx.y][threadIdx.x]=B[(k0+threadIdx.y)*N+c];
    __syncthreads();
    #pragma unroll
    for(int k=0;k<T;k++) acc += As[threadIdx.y][k]*Bs[k][threadIdx.x];
    __syncthreads();
  }
  if(r<M && c<N) C[r*N+c]=acc;
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

data reuse and tensor-core utilization.

Optimization direction

tile A/B in shared memory; accumulate in registers; vectorize stores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry
Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Llama Transformer Block

$$y = block(x; parameters, state)$$



What the topic does

Llama Transformer Block maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 120 · ATTENTION

Hard

TurboQuant KV Cache Attention

$$O = \text{softmax}(Q\ K^T / \sqrt{d} + \text{mask})\ V$$



What the topic does

TurboQuant KV Cache Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Multi-Head Latent Attention Decode

$$O = \text{softmax}(Q K^T / \sqrt{d} + \text{mask}) V$$



What the topic does

Multi-Head Latent Attention Decode maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 122 · MODEL

Hard

Diffusion Transformer Block

$$y = block(x; parameters, state)$$



What the topic does

Diffusion Transformer Block maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

Fused sequence of GEMM, normalization and elementwise kernels. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = model_block(x, state)
```

CUDA core kernel / structural sketch

```
__global__ void fused_block(const half* x,const half* w,half* y){
  // Load activation tile, compute projection, apply activation/norm,
  // add residual, and store once. Production versions use multiple CTAs.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

launch overhead and intermediate tensors.

Optimization direction

fuse residual/norm/activation; persistent weights; batch requests.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Non-Maximum Suppression

$$y[i] = f(x[i])$$



What the topic does

Non-Maximum Suppression maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

One output element per thread. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
y = op(x)  # elementwise / indexing operation
```

CUDA core kernel / structural sketch

```
__global__ void kernel(const float* x, float* y, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = op(x[i]);
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

memory bandwidth.

Optimization direction

coalesced contiguous loads; fuse adjacent elementwise work.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

PROBLEM 124 · ATTENTION

Hard

Block-Sparse KV Selection Attention

$$O = \text{softmax}(Q\ K^T / \sqrt{d} + \text{mask})\ V$$



What the topic does

Block-Sparse KV Selection Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Hard

Multi-Scale Deformable Attention

$$O = \text{softmax}(Q\ K^{\wedge}T / \text{sqrt}(d) + \text{mask})\ V$$



What the topic does

Multi-Scale Deformable Attention maps a mathematical tensor, sequence, image, graph or model operation onto many parallel GPU lanes. The correctness contract is defined by shapes, indexing, boundary rules and numerical precision.

Thread/block mapping

A CTA processes query rows against K/V tiles. A warp should access adjacent data when possible; blocks cooperate only through shared memory and barriers.

PyTorch correctness reference

```
# PyTorch
O = torch.nn.functional.scaled_dot_product_attention(Q, K, V, attn_mask=mask)
```

CUDA core kernel / structural sketch

```
__global__ void tiled_attention(const half* Q,const half* K,const half* V,half* O){
  // Load one Q tile and stream K/V tiles through shared memory.
  // Update running row max m and normalizer l (online softmax).
  // Accumulate P@V in registers, then normalize and store O.
}
```

Hardware-level execution

The CPU launches a grid. The GPU assigns resident blocks to SMs. Warp schedulers issue ready warps; loads travel through L2/L1 or shared memory into registers. Arithmetic runs on CUDA cores or matrix units, then stores return through cache to HBM.

Main bottleneck

HBM traffic, online softmax, long context.

Optimization direction

Flash-style tiling; keep statistics in registers; avoid materializing scores.

PyTorch vs CUDA comparison

View	PyTorch	CUDA
Abstraction	Tensor operation and autograd semantics	Explicit lanes, memory, barriers and launch geometry

Best use	Correctness, composition, training code	Learning internals and specialized fusion
Validation	Compare shapes, tolerances, edge cases, NaN/Inf behavior and multiple random sizes.	

Checkpoint: Explain the indexing, identify every global-memory read/write, estimate work per output, and state why the kernel is bandwidth- or compute-limited.

Final optimization checklist

- Start with a trusted PyTorch reference and randomized tests.
- Confirm contiguous dimensions and coalesced accesses.
- Move reused data into registers/shared memory.
- Minimize global synchronization and atomics.
- Use stable formulas and appropriate accumulation precision.
- Fuse only when it reduces traffic without excessive register pressure.
- Benchmark warm kernels across realistic shapes, not a single size.